SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

MINI PROJECT REPORT

Design and Development of Neon Snake Game

Using React, Vite, and Phaser

Course Code & Name:

BCSE203E — Web Programming

Faculty Name:

Dr. Komathy C

Team Members

V.C.Pavitran – 24BCE2019

Vellore Institute of Technology, Vellore

Vellore – 632 014

Winter Semester 2025-26

Submission Date: April 02, 2026

Abstract

This project presents the design and implementation of an interactive browser-based snake game titled “Neon Snake”. The application is developed using React and Vite for frontend architecture and Phaser as the primary game engine responsible for rendering, physics simulation, and real-time gameplay mechanics. The project explores how modern web technologies can be used to create responsive and visually engaging game environments directly within a browser.

The game introduces an open-world snake environment where the player navigates a continuously moving snake through a dynamic environment filled with enemies, food particles, powerups, and AI-controlled snakes. The design integrates multiple subsystems including enemy AI management, particle effects, procedural world generation, physics-based movement, and level progression.

This report discusses the system architecture, design decisions, algorithms, and implementation strategies used in building the project. Special attention is given to modular game design, performance optimization, and scalable game logic using JavaScript modules.

Table of Contents

S.No.	Sub - Headings		Page
1	Introduction		4
	1.1	Project Overview	
	1.2	Problem Statement	
	1.3	Objectives	
2	System Analysis		5
	2.1	Existing System	
	2.2	Proposed System	
	2.3	Advantages	
3	System Design		6-9
	3.1	Architecture Overview	6
	3.2	Data Flow Diagram	6
	3.3	Enemy AI Flowchart	7
	3.4	Perlin Noise Advantage	8
4	Implementation		10-20
	4.1	Game Initialization (GameCanvas.jsx)	10
	4.2	Snake Mechanics (Snake.js / SnakeBase.js)	11
	4.3	Enemy AI System (EnemyManager.js & EnemySnake.js)	12
	4.4	Food Management (FoodManager.js)	13
	4.5	Particle Effects (ParticleManager.js)	14
	4.6	Level Config(LevelConfig.js)	16
	4.7	Perlin Noise(PerlinNoise.js)	17
5	Results / Output Screens		21-23
6	Conclusion		24
7	Future Scope		24
8	References		24

1. Introduction

1.1 Project Overview

The Neon Snake project is an interactive arcade-style game implemented as a browser application using modern web development technologies. The project combines the React UI framework with the Phaser game engine to create a structured, modular, and scalable game architecture. React is used primarily to manage application structure and lifecycle, while Phaser manages rendering, physics simulation, and real-time game logic.

1.2 Problem Statement

Traditional snake games are usually simple grid-based games with limited interaction and static gameplay mechanics. The objective of this project is to design a modern version of the snake game that introduces dynamic enemy behavior, open world exploration, particle effects, and powerup mechanics while maintaining smooth performance within a browser environment.

1.3 Objectives

- Design an interactive browser-based snake game.
- Integrate React with Phaser for modular architecture.
- Implement AI-controlled enemy snakes.
- Implement dynamic food spawning and powerup mechanics.
- Provide multiple levels with different gameplay difficulty.
- Create optimized rendering and particle effects.

2. System Analysis

2.1 Existing System

Traditional snake games operate in a constrained grid environment where the snake moves in discrete directions and the gameplay logic is relatively simple. Enemy AI is typically absent, and the player only interacts with randomly generated food items.

2.2 Proposed System

The proposed system introduces an open-world snake game with advanced gameplay mechanics. The player interacts with multiple entities including enemy snakes, food particles, animals, and powerups. The system is implemented using modular JavaScript components where each module handles a specific aspect of gameplay.

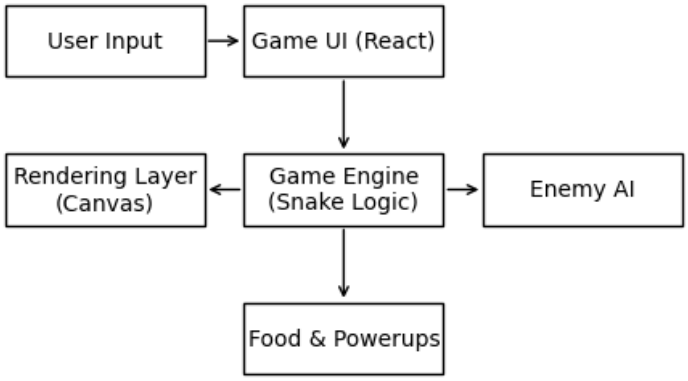
2.3 Advantages

- Dynamic enemy AI behavior
- Modular architecture for scalability
- Advanced visual effects using particle systems
- Multiple difficulty levels
- Optimized memory management using pooled particle emitters

3. System Design

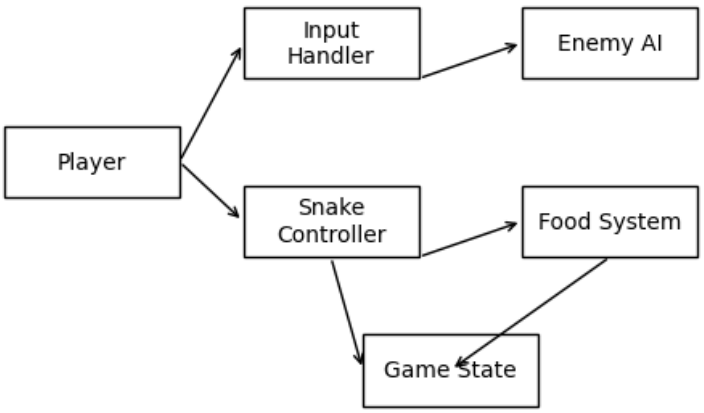
3.1 Architecture Overview

The system follows a modular architecture where individual components manage specific gameplay features. The main application initializes the React environment which then embeds the Phaser game instance inside a canvas container.



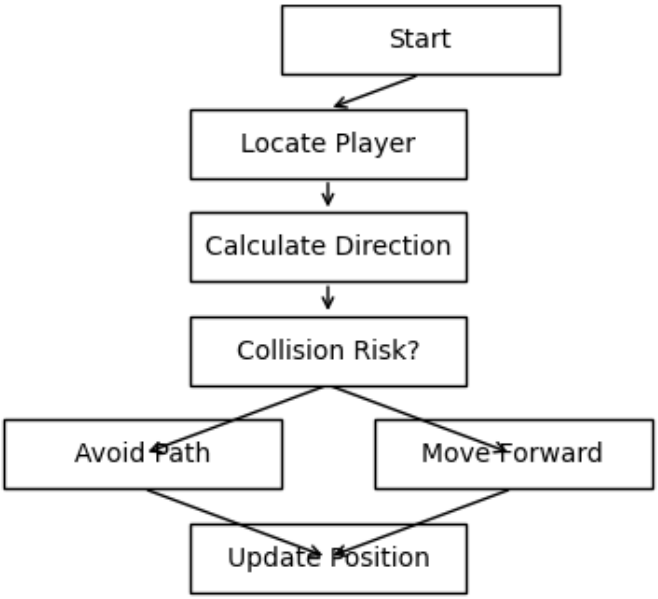
3.2 Data Flow Diagram (DFD)

The data flow diagram represents how information moves between modules such as the player snake, enemy manager, food manager, particle system, and level manager.



3.3 Enemy AI Flowchart Perlin Noise

The enemy snake uses a simple decision-based AI system that tracks the player's location and attempts to intercept or block the player's movement path.



3.4 Perlin Noise Advantage

In procedural game environments, generating terrain using simple random numbers often produces chaotic and unrealistic patterns. Adjacent positions may have completely unrelated values, which leads to abrupt transitions and visually unnatural landscapes. To overcome this limitation, the project utilizes the **Perlin Noise algorithm**, a gradient-based noise generation technique widely used in computer graphics and procedural world generation.

Perlin Noise produces **smooth and continuous variations between neighboring points**, allowing terrain features such as hills, valleys, and elevation gradients to emerge naturally. The algorithm works by assigning pseudo-random gradient vectors to grid points and interpolating between them using smooth fade functions. This interpolation ensures that nearby coordinates produce similar values, resulting in coherent terrain patterns instead of random spikes.

Additionally, the implementation incorporates **Fractal Brownian Motion (FBM)**, which layers multiple frequencies of Perlin Noise to generate richer and more detailed terrain structures. Lower frequency layers produce large terrain features such as hills, while higher frequency layers introduce finer variations such as surface roughness. This multi-octave

approach allows the system to simulate natural terrain complexity while maintaining computational efficiency.

Using a **seeded noise generator** further ensures that each game session can produce a unique world while still allowing reproducibility. The same seed will always generate the same terrain layout, which is useful for debugging, level sharing, and deterministic world generation.

4. Implementation

4.1 Game Initialization (GameCanvas.jsx)

The **GameCanvas component** is the central controller of the game. It is responsible for initializing the canvas rendering context, handling the main game loop, and coordinating interactions between the player snake, enemy snakes, particles, and level configuration.

The component uses React hooks such as **useEffect**, **useRef**, and **useState** to maintain the game state and update the canvas continuously.

```
import React, { useRef, useEffect } from "react";

import ParticleManager from "./ParticleManager";

import { LEVEL_CONFIG } from "./LevelConfig";


function GameCanvas() {

 const canvasRef = useRef(null);

 const particleManager = useRef(new ParticleManager());

 let snake = [{ x: 10, y: 10 }];

 let direction = { x: 1, y: 0 };

 useEffect(() => {

  const canvas = canvasRef.current;

  const ctx = canvas.getContext("2d");

  function gameLoop() {

   updateGame();

   renderGame(ctx);

   requestAnimationFrame(gameLoop);

  }

  gameLoop();

 }, []);

 function updateGame() {
```

```
  const head = { ...snake[0] };


  head.x += direction.x;
  head.y += direction.y;
  snake.unshift(head);
  snake.pop();
  particleManager.current.update();
 }
 function renderGame(ctx) {
  ctx.clearRect(0, 0, 800, 600);
  snake.forEach(segment => {
   ctx.fillRect(segment.x * 20, segment.y * 20, 20, 20);
  });
  particleManager.current.render(ctx);
 }
 return <canvas ref={canvasRef} width={800} height={600} />;
}

export default GameCanvas;
```

4.2 Snake Mechanics (Snake.js / SnakeBase.js)

The **snake mechanics module** is responsible for controlling the player snake's movement, growth, and collision detection. The system maintains the snake as a sequence of connected segments stored in an array. Each frame, the snake moves forward by creating a new head position based on the current direction vector while optionally removing the tail segment unless food has been consumed.

```
function updateSnake() {
  const head = { ...snake[0] };
  // Move the snake in the current direction
  head.x += direction.x;
  head.y += direction.y;
  // Add new head to the front of the snake
  snake.unshift(head);
  // Check if food was eaten
  if (head.x === food.x && head.y === food.y) {
    score += 10;
    spawnFood();
  } else {
    // Remove the tail if food not eaten
    snake.pop();
  }

}
```

4.3 Enemy AI System (EnemyManager.js & EnemySnake.js)

The **enemy AI module** introduces autonomous snakes that attempt to intercept the player. Each enemy snake evaluates its environment and chooses actions such as fleeing from larger snakes, hunting smaller snakes, or searching for food. The decision making logic uses distance calculations and behavioral thresholds. The AI continuously analyzes the player's position and adjusts its direction accordingly. This approach creates dynamic gameplay by forcing the player to avoid enemy paths.

```
function updateEnemy(enemy, player) {
  let dx = player.x - enemy.x;
  let dy = player.y - enemy.y;
  if (Math.abs(dx) > Math.abs(dy)) {
    enemy.direction = {
      x: dx > 0 ? 1 : -1,
      y: 0
    };
  } else {
    enemy.direction = {
      x: 0,
      y: dy > 0 ? 1 : -1
    };
  }
  enemy.x += enemy.direction.x;
  enemy.y += enemy.direction.y;
}
```

4.4 Food Management (FoodManager.js)

The **FoodManager module** controls the generation and placement of food items within the game environment. It ensures that food appears randomly within valid grid locations and prevents spawning on top of snake segments.

This function generates food at random positions within the grid while ensuring that the food does not overlap with any segment of the snake. The algorithm repeatedly generates candidate positions until a valid empty cell is found. This guarantees fair gameplay and prevents impossible scenarios where food spawns inside the snake body.

Instead of giving each food item its own animation, a single '_pulse' object is called. Every frame, the manager iterates through all active food and sets their scale to match this single value. This reduces the number of active tweens from hundreds down to one.

```
function spawnFood() {
 let validPosition = false;
 let newFood;
 while (!validPosition) {
  newFood = {
   x: Math.floor(Math.random() * gridWidth),
   y: Math.floor(Math.random() * gridHeight)
  };
  validPosition = !snake.some(segment =>
   segment.x === newFood.x && segment.y === newFood.y
  );
 }
 food = newFood;
}
```

4.5 Particle Effects (ParticleManager.js)

The **ParticleManager module** manages particle effects used to enhance gameplay visuals. These particles appear during events such as food collection, enemy destruction, or level transitions.

Particles are lightweight graphical objects with properties such as **position, velocity, lifespan, and color**.

```
class ParticleManager {
 constructor() {
  this.particles = [];
 }
 spawn(x, y, color) {
  for (let i = 0; i < 10; i++) {
   this.particles.push({
    x: x,
```

```
        y: y,
        vx: (Math.random() - 0.5) * 4,
        vy: (Math.random() - 0.5) * 4,
        life: 60,
        color: color
      });
    }
  }
  update() {
    this.particles.forEach(p => {
      p.x += p.vx;
      p.y += p.vy;
      p.life--;
    });
    this.particles = this.particles.filter(p => p.life > 0);
  }
  render(ctx) {
    this.particles.forEach(p => {
      ctx.fillStyle = p.color;
      ctx.fillRect(p.x, p.y, 3, 3);
    });
  }
}
export default ParticleManager;
```

4.6 Level System (LevelConfig.js)

The **LevelConfig module** defines configuration parameters for each level in the game. It allows developers to easily modify gameplay difficulty by adjusting properties such as enemy count, speed, food spawn rate, and map size.

This design follows a **data-driven approach**, separating gameplay parameters from logic implementation.

```
export const LEVEL_CONFIG = {
 level1: {
  snakeSpeed: 5,
  enemyCount: 1,
  foodSpawnRate: 1,
  mapSize: 20
 },
 level2: {
  snakeSpeed: 7,
  enemyCount: 2,
  foodSpawnRate: 2,
  mapSize: 25
 },
 level3: {
  snakeSpeed: 9,
  enemyCount: 3,
  foodSpawnRate: 3,
  mapSize: 30
 }
};
```

4.7 Perlin Noise (PerlinNoise.js)

The **PerlinNoise class** generates smooth random values used for **procedural terrain generation**. Instead of generating completely random numbers, Perlin Noise produces **smooth continuous variations**, which makes terrain look natural.

Structure of the Code

The file contains **six main parts**

1. Class initialization

   ```
   constructor(seed = Math.random()) {
    this._perm = this._buildPermutation(seed)
   }
   ```

   The constructor builds a **permutation table** using the provided seed.

2. Seeded random generator

   ```
   let s = Math.floor(seed * 0xffffffff) >>> 0
   ```

   This converts the seed into a **32-bit integer**.

   The function below is **Mulberry32**, a fast pseudo-random generator.

   ```
   const rand = () => {
    s += 0x6d2b79f5
    let t = s
    t = Math.imul(t ^ (t >>> 15), t | 1)
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61)
    return ((t ^ (t >>> 14)) >>> 0) / 0xffffffff
   }
   ```

   This creates **random numbers between 0 and 1**, but they are **deterministic based on seed**.

3. Permutation table creation

```
for (let i = 255; i > 0; i--) {
  const j = Math.floor(rand() * (i + 1));
  [p[i], p[j]] = [p[j], p[i]]
}
```

This random permutation is used to determine **gradient directions**.

4. Fade and interpolation functions

```
_fade(t) {
  return t * t * t * (t * (t * 6 - 15) + 10)
}
```

This is a **smoothing function**. Instead of linear interpolation, this curve smooths the transitions. This is what makes Perlin noise **look natural**.

```
_lerp(a, b, t) {
  return a + t * (b - a)
}
```

This blends between two values. This helps combine **multiple gradient contributions**.

5. Gradient calculation

```
_grad(hash, x, y)
```

This function determines the **gradient vector direction**.

```
const h = hash & 7
```

This extracts the **lowest 3 bits** of the hash value. These bits determine which **gradient direction** is used. Then dot product is calculated:

- gradient · distance vector

This produces the noise contribution for that corner.

6. Noise value calculation

   get(x, y)

   This function calculates **Perlin noise value at coordinate (x,y)**.

   Steps:

   Step 1 — Grid Cell

   const X = Math.floor(x) & 255
   const Y = Math.floor(y) & 255

   Find the **grid cell containing the point**.

   Step 2 — Local Position

   xf = x - Math.floor(x)
   yf = y - Math.floor(y)

   This calculates the **position inside the grid cell**.

   Example: x = 5.3 → xf = 0.3

   Step 3 — Fade smoothing

   u = fade(xf)
   v = fade(yf)

   These values smooth interpolation.

   Step 4 — Gradient hash lookup

   aa = p[p[X] + Y]
   ab = p[p[X] + Y + 1]
   ba = p[p[X + 1] + Y]
   bb = p[p[X + 1] + Y + 1]

   This finds gradient vectors for the **four corners of the cell**.

   aa ba
   ab bb

Step 5 — Interpolation

```
return lerp(
  lerp(grad(aa...), grad(ba...), u),
  lerp(grad(ab...), grad(bb...), u),
  v
)
```

This blends the four gradients to produce a **smooth value between -1 and 1**.

7. Fractal Brownian Motion (FBM)

   fbm(x,y,octaves)

   This creates **more detailed terrain** by layering multiple noise frequencies.

   Code:

   ```
   val += this.get(x * freq, y * freq) * amp
   ```

   Each octave:

   - frequency doubles
   - amplitude halves

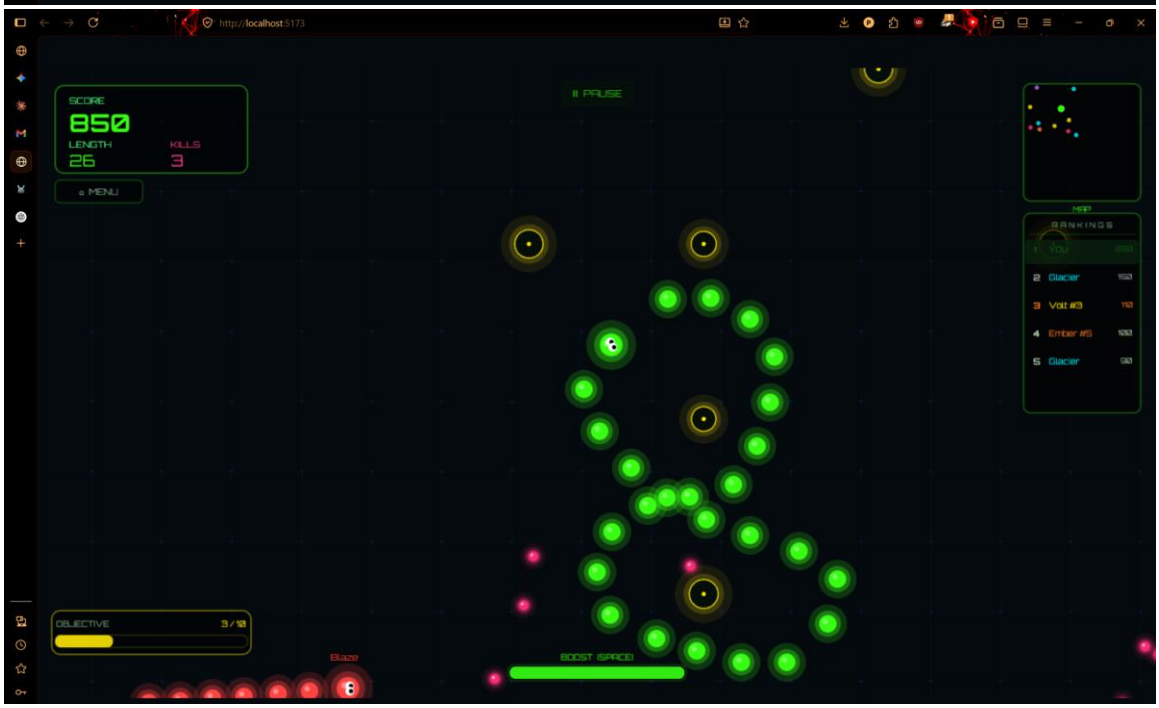
   Final value is normalized to:

   - $0 \rightarrow 1$

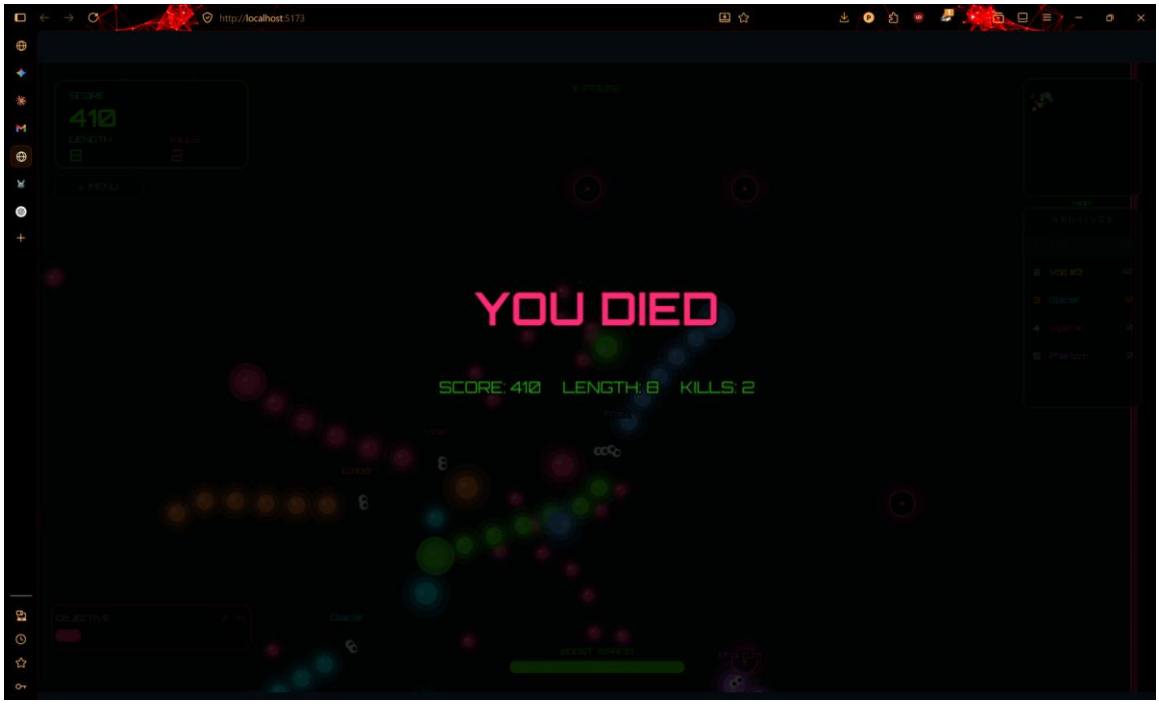
5. Results / Output Screens

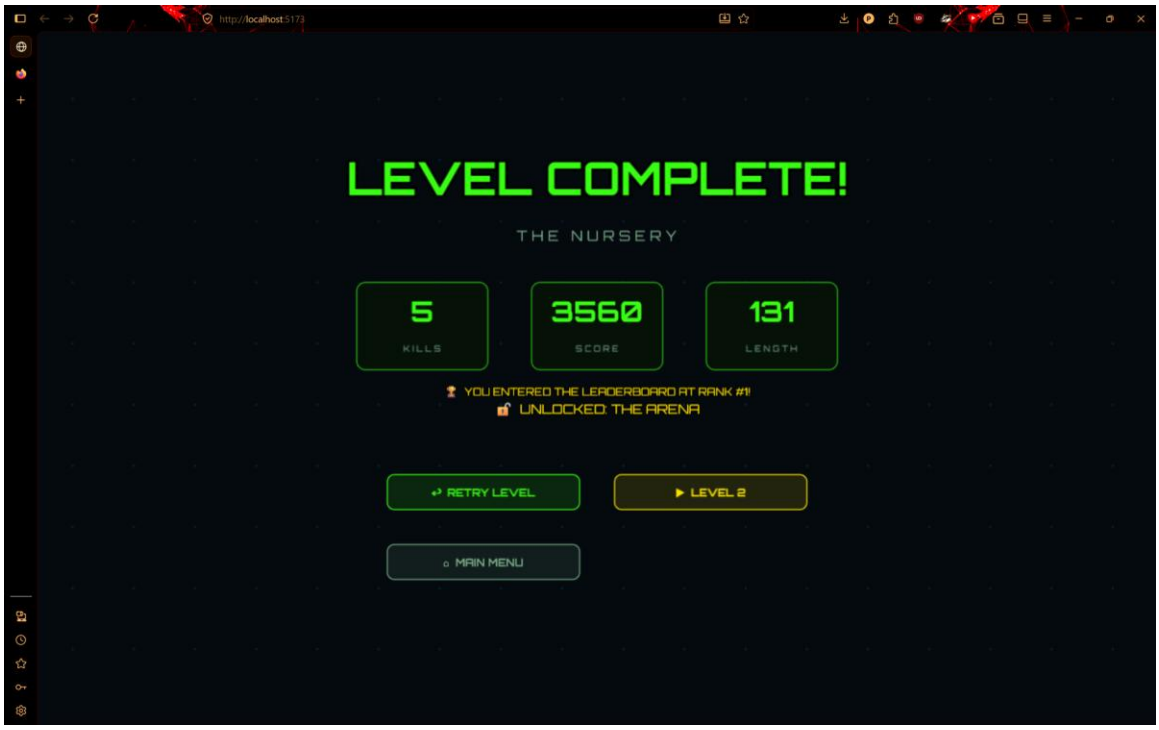
The implemented system successfully demonstrates smooth gameplay and responsive controls.






http://localhost:5173
SCORE
1420
LENGTH
9
KILLS
7
⏸ PAUSE
⌂ MENU
RANKINGS
1 YOU
2 Phantom
3 Blaze #1
4 Volt #1
5 Toxin #1
Blaze
Blaze
Volt
Toxin
Phantom
OBJECTIVE
BOOST (SPACE)


http://localhost:5173
YOU DIED
SCORE: 410 LENGTH: 8 KILLS: 2


http://localhost:5173
LEVEL COMPLETE!
THE NURSERY
5
KILLS
3560
SCORE
131
LENGTH
YOU ENTERED THE LEADERBOARD AT RANK #1!
UNLOCKED: THE ARENA
RETRY LEVEL
LEVEL 2
MAIN MENU


http://localhost:5173
LEADERBOARD
TOP SCORES ACROSS ALL LEVELS
PLAYER SCORE KILLS LENGTH LEVEL DATE
1 PV 3310 10 84 L2 THE ARENA
2 PV 3150 15 20 L3 THE PIT
3 PV 3070 15 14 L3 THE PIT
4 PV 2730 13 19 L3 THE PIT
5 PV 2730 13 19 L3 THE PIT
6 PV 2360 11 11 L3 THE PIT
7 PV 2360 11 11 L3 THE PIT
8 PV 2360 11 11 L3 THE PIT
9 PV 2360 11 11 L3 THE PIT
10 PV 2360 11 11 L3 THE PIT
BACK TO MENU
CLEAR SCORES

6. Conclusion

The Neon Snake project demonstrates how modern web technologies can be used to develop sophisticated interactive games directly within the browser. The integration of React and Phaser enables a structured approach to game development while maintaining performance and modularity.

7. Future Scope

Future improvements may include multiplayer mode, different designs for snakes, improvement in UI/UX, reduction of latency, terrain generation in map using perlin noise, mobile phone support, stable framerate support through different displays, high fps support, improved graphics support, addition of 3D world which will be a different mode available as an experimental feature and using three.js to improve 3D graphics in that mode.

8. References

• Phaser Game Engine Documentation

• React Documentation

• JavaScript Game Development Resources